Informational Latency

Board-B setpoint echo test — latency_bag1, 14.8 s, four step commands

The answer to the question you set

The **information path costs 80–112 ms**. Measured two ways that agree: the per-edge dead time over the four steps is **83 ms** (78–95), and fitting the whole run as the command through diff_drive_controller's acceleration limiter, delayed, gives **L = 112 ms at r² = 0.9990**.

Against the **254–274 ms** the real wheels took to respond in the iteration 3 runs, measured the same way, that leaves roughly **150–170 ms for the PID loop, the motors and inertia**. So the split is about **40 % informational, 60 % physical**, and the bigger half is the half you can tune.

One thing the test exposed that was not on the list: on a step input, **192 ms of the apparent delay is the 1.5 rad/s² acceleration limit** in `diffbot_controllers.yaml`, not latency at all. The previous report's correlation-peak figures were partly measuring that.

Your `velocity_rolling_window_size` change from 10 to 3 landed exactly as predicted: **34 ms measured against 33 ms predicted**, down from 154.

1. What the echo test actually gives us

With board B reporting its commanded setpoint back as the measured velocity, `/dynamic_joint_states` no longer contains wheel motion. It contains the command, having made the round trip host → serial → board A → board B → back → `ros2_control` state interface. No motor, no PID, no inertia. Two checks that the channel is what it is supposed to be:

• the echo peaks at **0.9985** against a commanded 1.000;
• the published odometry divided by the raw wheel-velocity difference comes out at **0.1928**, against r/L = 0.165 / 0.856 = 0.1928 — so the URDF and wheel-separation corrections are in force and the scaling is exactly right.

You were right that propping the robot up does not matter here. It would matter if we were measuring motion; we are measuring a data path, and the motor is not in it.

One thing to know before reading any number in this report

Every one of the 462 `/diffbot_base_controller/cmd_vel` messages in this bag has `header.stamp` **= 0**. rqt's message publisher does not fill it in. So the command cannot be referenced to its own stamp and everything here uses the **bag receive time** instead.

The previous report used header stamps, where the command is already 32–69 ms old when it arrives. To keep the comparison honest, the iteration 3 bags were re-run on receive time as well, and those are the numbers quoted here — the IMU peak becomes 271 ms rather than the 327 ms in the old report. Same data, different reference point, not a correction.

If you want header stamps next time, rqt can fill them with an expression such as `auto` on the stamp field, or publish from a small script.

2. The figures you asked for

These are the same three views as the previous report, with the IMU dropped. The important caveat is in the first one.

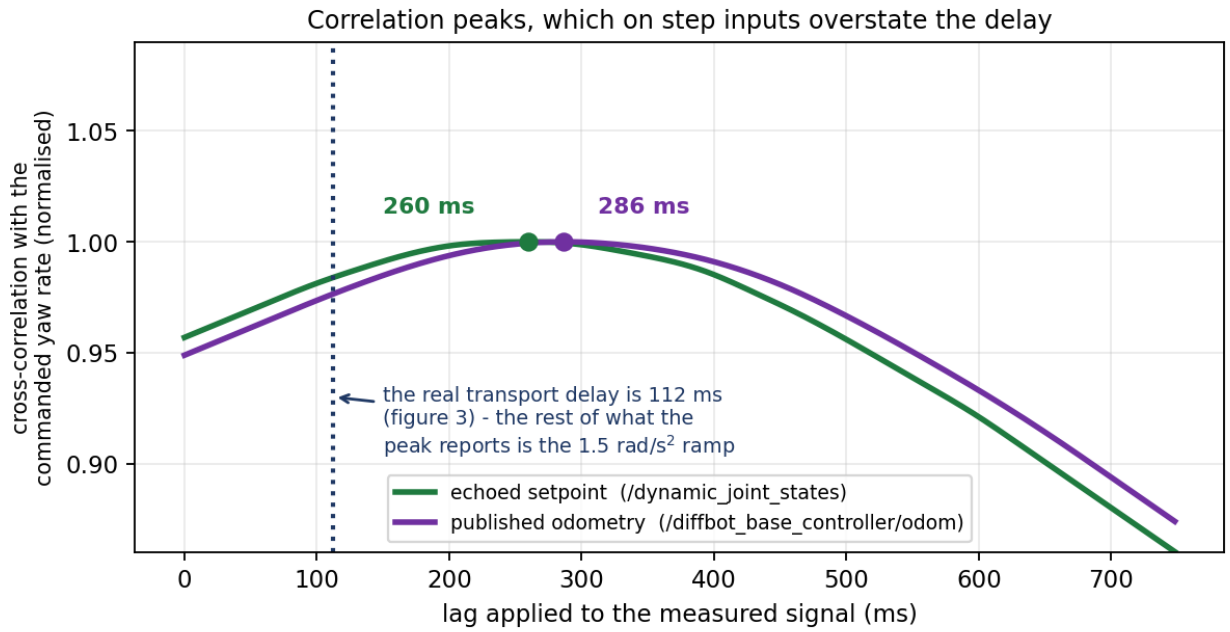


Figure 1. Cross-correlation against the commanded yaw rate. On a step input the peak folds the acceleration ramp in with the delay and reports roughly 2.3× too much. Use figure 3 for the delay; this one is here for continuity with the previous report.

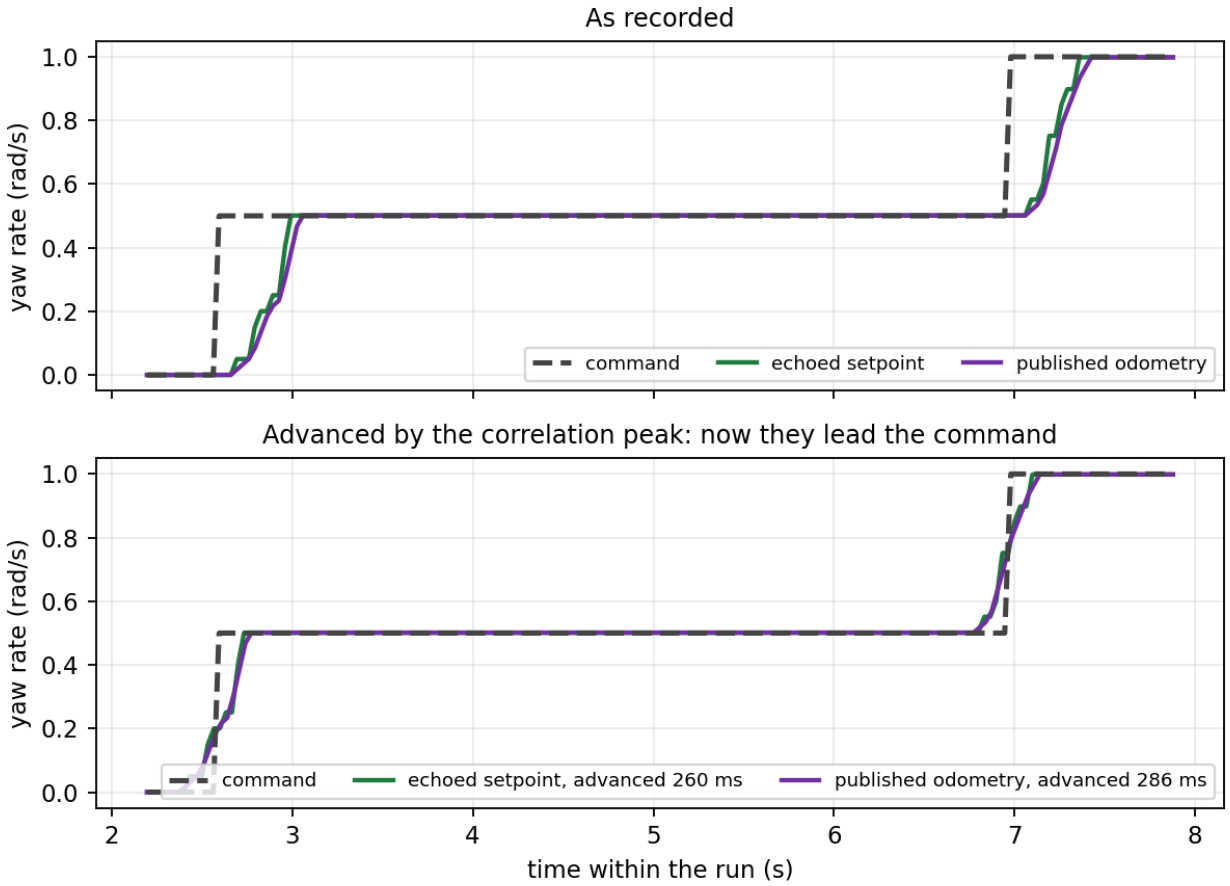


Figure 2. The first two steps. Advancing each trace by its own correlation peak over-corrects — they now start before the command does, which is the same point figure 1 makes: the response is a delay followed by a ramp, not a pure delay.

3. Separating the delay from the acceleration limit

`diff_drive_controller` rate-limits the command before it ever reaches the hardware. `angular.z.max_acceleration` is 1.5 rad/s² and `update_rate` is 30, so the setpoint can move only 1.5 / 30 = **0.05 rad/s per tick**. The echo moves in exact multiples of **0.0501 rad/s**, which is that quantum, and the measured ramp rates across the four steps are 1.60, 1.79, 1.55 and 1.79 rad/s². So the model to fit is:

```
echo(t) = ratelimit(command, 1.5 rad/s², 30 Hz)(t − L)
```

fit	L	r²
command as-is, pure delay only	304 ms	0.9728
command through the 1.5 rad/s² limiter	**112 ms**	**0.9990**

The limiter alone accounts for **192 ms** of what a naive fit calls dead time. And the per-edge measurement, which needs no model at all — just the first tick on which the echo moved — agrees with the smaller number:

command step	dead time	ramp	implied acceleration
0.0 → 0.5 rad/s	80 ms	312 ms	1.60 rad/s²
0.5 → 1.0 rad/s	95 ms	280 ms	1.79 rad/s²
1.0 → 0.2 rad/s	78 ms	516 ms	1.55 rad/s²
0.2 → 0.0 rad/s	78 ms	112 ms	1.79 rad/s²

Mean dead time **83 ms**, spread 78–95 ms, against a 30 Hz controller tick of 33 ms — so most of that spread is just which tick the command landed on.

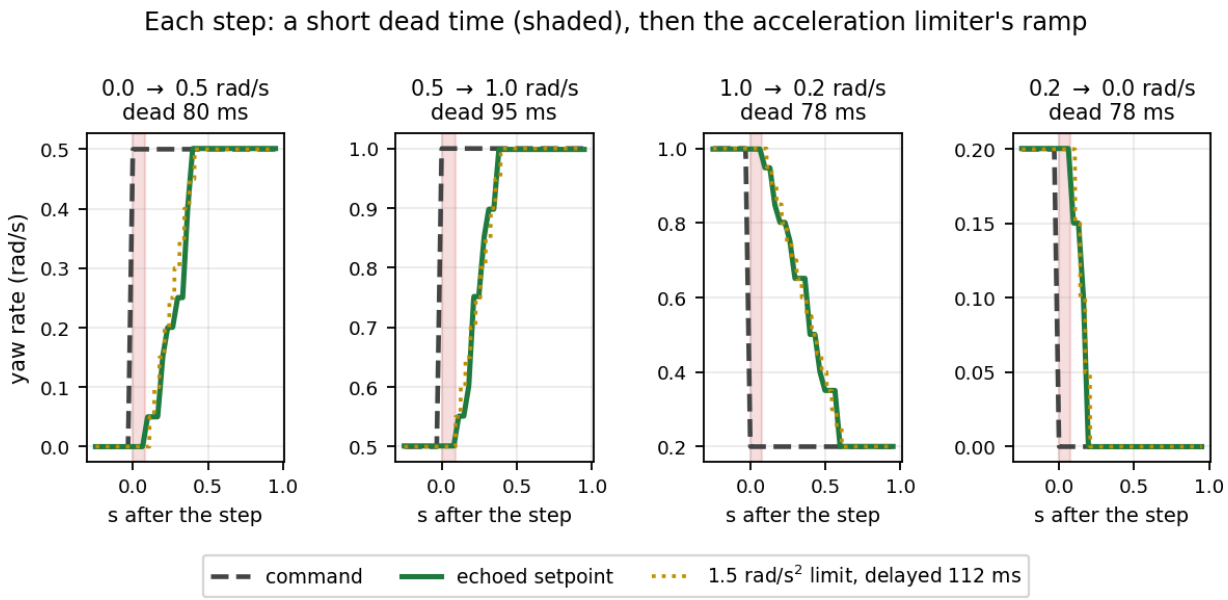


Figure 3. Each step with the rate-limiter model overlaid. The shaded band is the measured dead time; everything after it is the limiter's ramp, reproduced by the dotted line with a single 112 ms delay and no other free parameter.

4. The odometry smoothing, after your change

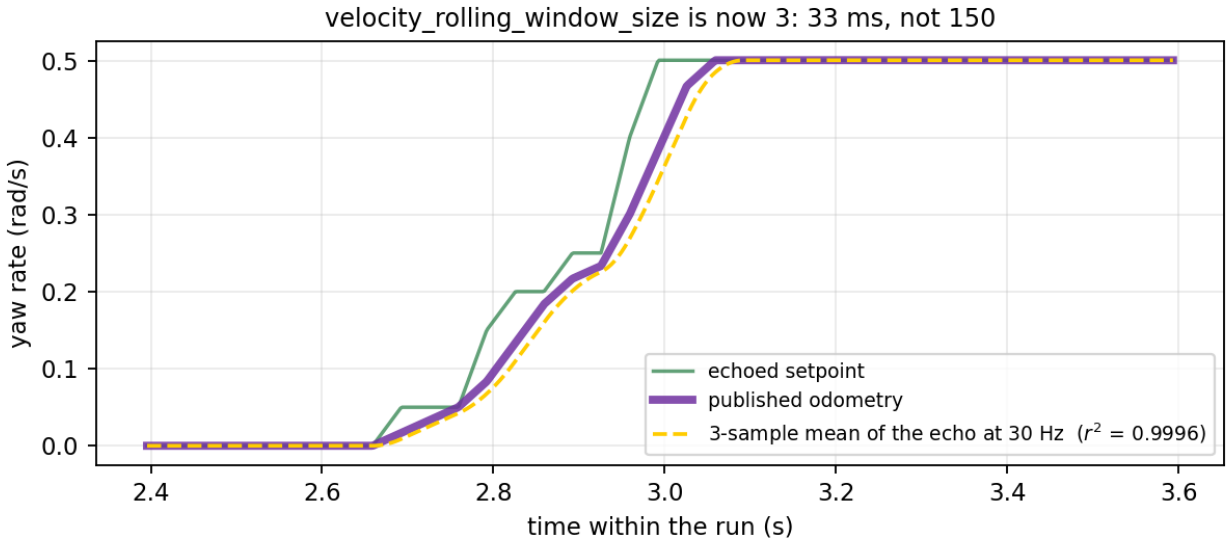


Figure 4. The published odometry against a 3-sample mean of the echo at 30 Hz, r² = 0.9996.

	window 10 (previous report)	window 3 (now)
predicted group delay	150 ms	33 ms
measured, odometry behind the wheel signal	154 ms	**34 ms**
residual after applying the mean	0 ms, r² 0.9997	0 ms, r² 0.9993

That is about as clean a confirmation as this kind of thing gets: the change removed 120 ms and the remaining 34 ms is what the arithmetic says a 3-sample window must cost. Nothing further to do here.

5. rtlatency

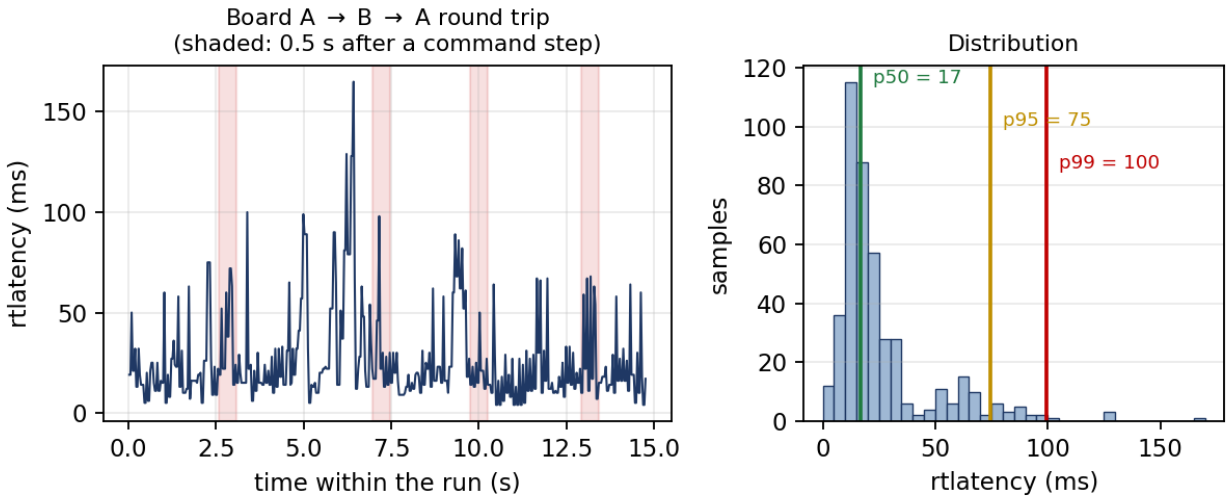


Figure 5. The board A → B → A round trip reported by the firmware, over the whole run and as a distribution. The robot stayed in autonomous mode throughout, so nothing has been filtered out.

p05	p50	p90	p95	p99	max
5 ms	**17 ms**	61 ms	75 ms	100 ms	165 ms

The median 17 ms is a modest share of the 83–112 ms information path, but the tail is not: one sample in a hundred exceeds 100 ms, which on its own would be most of the budget. It is also mildly worse exactly when it matters — in the 0.5 s after a command step the median rises to **22 ms** against 16 ms elsewhere.

Subtracting what we can account for leaves roughly **50–80 ms** unexplained between the host and board B: the serial write from the host, board A's own processing, the read scheduling inside the `ros2_control` loop, and the state-interface publish. The 30 Hz `controller_manager` tick contributes 0–33 ms of that, averaging about 17 ms.

6. Putting it against the real-motion runs

Fitting the iteration 3 bags the same way — receive-time reference, same limiter model, against the real wheel velocities rather than an echo:

	transport + response (L)	r^2	limiter's share
echo test (no motor in the loop)	**112 ms**	0.9990	192 ms
real_bag7, real motion	274 ms	0.9825	16 ms
real_bag9, real motion	254 ms	0.8692	28 ms

Note how little the limiter contributes during normal driving: nav2 asks for more than 1.5 rad/s² in only **0.3–0.9 %** of command samples, so it almost never binds on the course. The 192 ms it costs here is an artefact of step inputs, not something the robot normally pays.

How much confidence to put on the 150–170 ms

The **112 ms information figure is solid** — two independent methods, r^2 0.999, and no motor in the loop for load or tyre grip to confuse.

The **physical figure is a difference between two differently-excited tests**: steps with the wheels in the air, against smooth nav2 commands under load on the ground. The fit quality on real motion is also lower (r^2 0.87–0.98, because real wheels do more than follow a ramp). Treat 150–170 ms as indicative, good to perhaps ±30 ms, not as a calibrated constant.

The clean way to tighten it would be the same step test with the echo reverted and the robot on the ground: identical inputs, identical analysis, and the difference is then the motor response with nothing else changing.

7. What this changes

	what	why
1	The PID loop and the drivetrain are now the bigger half, so that is where the remaining time is.	Roughly 150–170 ms against 112 ms of information path. Before this test it was reasonable to suspect the link; it is now measured, and the link is the smaller share.
2	Reconcile `angular.z.max_acceleration` (1.5) with MPPI's `az_max` (2.0).	Unchanged from the last report and now quantified: on a step it costs 192 ms. It binds on under 1 % of normal commands, so this is about MPPI predicting its own trajectories correctly rather than about speed — but the two numbers should still agree.
3	Chase the `rtlatency` tail, not its median.	17 ms typical is fine. p99 of 100 ms and a 165 ms maximum are not, and they cluster after command changes.
4	Leave `velocity_rolling_window_size` at 3.	Measured 34 ms against 33 predicted. It did what it was supposed to do.
5	Next bag: fill in the command's header stamp.	It would remove the one place this report has to hedge, and it is a one-line change in whatever publishes the test commands.

Tooling: `claude_tools/step_latency.py` produces the per-edge and model numbers, `claude_tools/latency_breakdown.py --cmd-recv` the comparison against the iteration 3 bags, and `claude_tools/make_figures_informational.py` the figures.